

Report on Real-Time Document Editing System Implementation

Introduction

The Real-Time Document Editing System is designed to allow multiple users to collaboratively edit a document in real-time. This report outlines the implementation details, architecture, functionalities, and any quirks observed during development. The system is built using Python, leveraging gRPC for communication, and utilizes the `curses` library for a terminal-based user interface.

Architecture

The system consists of three main components:

1. **Server:** Responsible for managing document state, processing client requests, and broadcasting updates to all connected clients.
2. **Logger Server:** Logs changes made to the document, ensuring that all edits are recorded for auditing and debugging purposes.
3. **Clients:** Terminal-based interfaces that allow users to edit the document, displaying real-time updates and sending editing commands to the server.

Folder Structure

The project is organized into the following directory structure:

bash

Copy code

```
/project-root
├── protofiles/
│   └── document.proto
├── clients/
│   └── client.py
├── server/
│   ├── server.py
│   └── logger_server.py
└── toBuild.sh
```

Protocol Buffers

The communication between clients and servers is defined using Protocol Buffers (protobufs). The `document.proto` file outlines the necessary messages and services, including document editing operations and synchronization requests.

Implementation Details

1. Server

The `server.py` script handles client connections and manages the document state. It includes the following key components:

- **Initialization:** The server starts by loading the document and preparing to accept client connections.
- **Client Handling:** Each client connection is handled in a separate thread or coroutine. This allows multiple clients to edit the document simultaneously.
- **Editing Operations:** The server processes `insert` and `delete` operations, updating the document accordingly and notifying all clients of changes.
- **Synchronization:** The server continuously syncs document changes to all connected clients, ensuring that each client has the latest version of the document.

2. Logger Server

The `logger_server.py` script is designed to log all document changes. Key features include:

- **Logging Changes:** The logger server receives edit requests from clients and appends these changes to a log file.
- **Asynchronous Logging:** The logging operations are handled asynchronously to avoid blocking the main document server.

3. Client

The `client.py` script provides a terminal-based interface for users to edit the document. Key functionalities include:

- **Curses Interface:** The client utilizes the `curses` library to create a user-friendly text interface, allowing users to navigate and edit the document easily.
- **Editing Commands:** Users can perform basic editing operations such as inserting and deleting characters. The interface updates in real-time as changes are made.
- **Error Handling:** The client includes error handling for situations where the server is unreachable or crashes, providing feedback to the user.

4. Bash Build Script

The `toBuild.sh` script automates the setup process for the project, including:

- Installing necessary dependencies (`grpcio` and `grpcio-tools`).
- Compiling protobuf files to generate Python classes.
- Starting the logger server, main server, and multiple client instances in separate terminal windows.

Functionality

The system meets the following functionalities:

- **Real-Time Collaboration:** Multiple clients can edit the same document simultaneously, with changes reflecting in real-time across all clients.
- **Basic Document Editing:** Users can insert and delete characters, with changes logged and synchronized.
- **Error Handling:** The system gracefully handles errors related to server unavailability, providing users with appropriate feedback.
- **Change Logging:** All document changes are logged by the logger server, allowing for audit trails and debugging.

Quirks and Known Issues

1. **Document Synchronization:** The implementation may experience slight delays in document synchronization due to network latency.
2. **Error Handling:** Clients cannot automatically reconnect if the server crashes; users must restart the clients manually.
3. **Document Size:** Very large documents may lead to performance degradation.
4. **Curses Interface:** The `curses` library may have limitations based on terminal settings, which can affect the user experience.
5. **Limited Input Handling:** The current implementation supports basic text editing only; advanced features are not yet available.
6. **Terminal Compatibility:** The use of `gnome-terminal` means the setup is primarily for Linux environments.

Conclusion

The Real-Time Document Editing System successfully implements a collaborative editing environment using gRPC and terminal interfaces. While the system meets its core functionalities, there are several quirks and limitations to consider. Future improvements may include enhanced input handling, automatic reconnection capabilities, and additional editing features to improve the user experience.